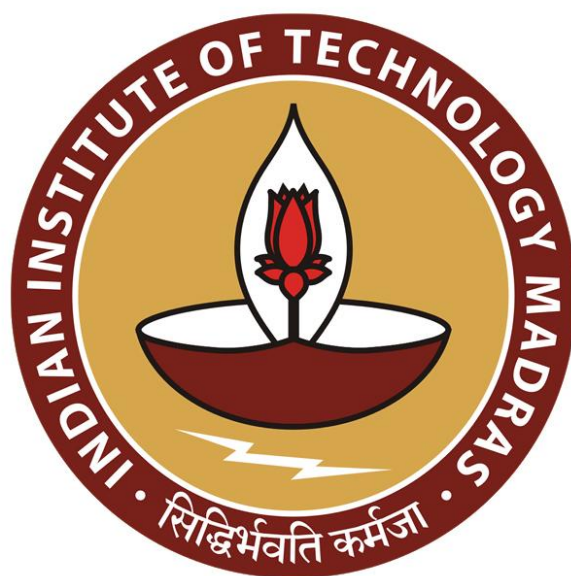


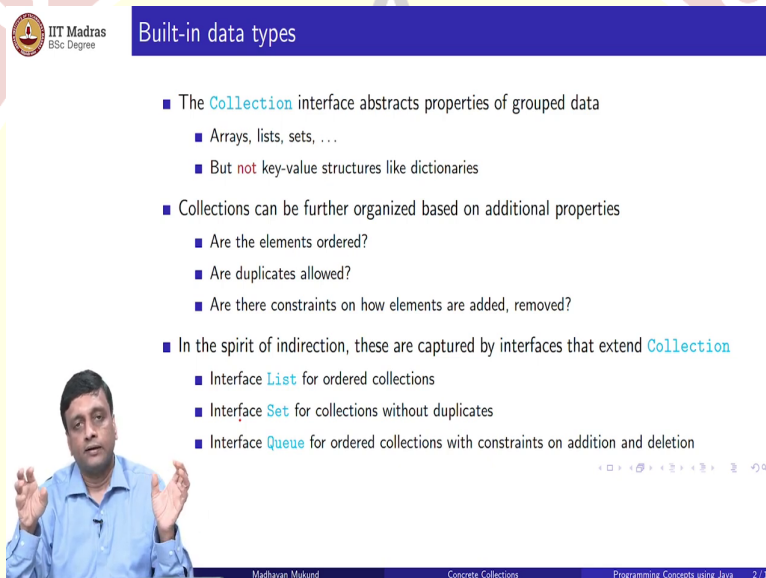


IIT Madras
ONLINE DEGREE

Programming Concept using JAVA
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Concrete Collections

So, we have seen the concrete, not the concrete, so we have seen the Collection interface, which is an abstract interface, which has various parts to it. And we have seen how this interface is then implemented by this AbstractCollection interface, which tells us how to implement many of these things by default.

(Refer Slide Time: 00:32)



Built-in data types

- The **Collection** interface abstracts properties of grouped data
 - Arrays, lists, sets, ...
 - But **not** key-value structures like dictionaries
- Collections can be further organized based on additional properties
 - Are the elements ordered?
 - Are duplicates allowed?
 - Are there constraints on how elements are added, removed?
- In the spirit of indirection, these are captured by interfaces that extend **Collection**
 - Interface **List** for ordered collections
 - Interface **Set** for collections without duplicates
 - Interface **Queue** for ordered collections with constraints on addition and deletion

Madhavan Mukund Concrete Collections Programming Concepts using Java 2/10

So, now let us look at concrete collections. So, Collection itself captures all these groups, which are not key value pairs. So, which are not structures like dictionary, so arrays, lists, sets, and so on. And then what we can now observe is that within this there are categories. So, lists and arrays have an order there is the first element, the second element and so on. Whereas sets are unordered. So, there is a distinction between arrays and sets or lists and sets. Sets are supposed to be unique valued. You cannot have two elements of the same value in the Set. So, Set has only one of each type. So, no duplicates are allowed. This is not the case in an array or a list, the same value could appear at multiple positions. Something like a Queue has constraints on how you can add and remove.

So, in a Set, you can add and remove anything. In a list or an array, also you can add and remove, you can change the position and so on. But in a Queue, you can only add it the rear,

and removed from the front. So, these are all now different extra properties, which all go along with the basic idea of a collection of values. So, the logical thing is to capture each of these through a sub interface. Just like you have a sub class, you can have a sub interface. So, there is a sub interface called list which extends Collection which talks about ordered collections, collections in which there is a sequence of values.

Then Set is a sub interface of collection, which talks about collections without duplicates, they may or may not have an order, but the main thing is you do not have two copies of the same value in it. So, the crucial thing is that sets do not have duplicates. And then for instance, we will look at one specific thing called Queue, which is an ordered collection which have these constraints on insert and delete, so the basic queue, a vanilla queue that we have has a constraint that you can add the back and you can remove the front but we will see other kinds of queues also.

(Refer Slide Time: 02:26)

The List interface

- An ordered collection can be accessed in two ways
 - Through an iterator
 - By position — **random access**
- Additional functions for random access

```
public interface List<E>
    extends Collection<E>{
    void add(int index, E element);
    void remove(int index);
    E get(int index);
    E set(int index, E element);
}
```

$l[i] = v$

Madhavan Mukund Concrete Collections Programming Concepts using Java 3 / 10

- An ordered collection can be accessed in two ways

- Through an iterator
- By position — **random access**

- Additional functions for random access

- **ListIterator** extends **Iterator**

- **void add(E element)** to insert an element before the current index
- **void previous()** to go to previous element
- **boolean hasPrevious()** checks that it is legal to go backwards

```
public interface List<E>
    extends Collection<E>{
    void add(int index, E element);
    void remove(int index);
    E get(int index);
    E set(int index, E element);

    ListIterator<E> listIterator();
}
```



Madhavan Mukund

Concrete Collections

Programming Concepts using Java

3/10

So, let us start with List. So, a list is a sequence. But now, when we have a sequence of values, there are two ways that we can identify an element in the sequence. One is to start at the beginning and go through an iterator and reach it. Or we can say I want the value at position *i*, this is called random access. So, in any sequence, I can do this, I can start at the beginning and walk seven steps. Or I can say I want the value at position seven. So, in this list interface, we have these extra random-access functions now.

So, remember that the original interface of collection had an add and an iterator. So, that allows us to go through the thing from beginning to end. But now that we have this notion of a position, it is meaningful to talk about the value at position *i*. So, I can say add an element *E*, at this index. So, this is a different add. This is not the add, which is there in the collection interface, this is a different add, that one just says add an element *E*, this one says add an element *E* at this position.

Similarly, I can say remove a value at this position. So, you know, shrink the sequence by removing the seventh value. I can also ask. This is a familiar thing that we normally write like this. I can say get me the value at an index. And similarly, I can say, set this value to be something. So, I give you the index position, and I say change this value. So, these are normal things that we do implicitly when we deal with lists and arrays. But now they are kind of changed into functions because they are in this object-oriented setting.

So, we can get a value at a position, set a value to a position, we can add a value at a position, we can remove value at a position. So, we now have an extra thing, which is that even the

iterator can be a little bit more functional in some sense, in the basic iterator, so the basic iterator hasNext. And then it says next gives us next. Now, if I have an iterator, remember that there is a notion of a position. So, I can take a position and I can remove it. Now, if I have something where I know these positions, then I can also insert something there. So, there is something called a list iterator.

A list iterator is something which extends the vanilla iterator that we have with collections. And what this list interface allows us is to call for a list iterator. So, remember, the collection gives us a normal iterator. So, we can still get a normal iterator if you want. But we can also ask for a list iterator. And the list iterator is more functional than a normal iterator. So, in addition to everything that normal iterator can do get me the next value and has a next value. Here, I can add an element at the position before the current index.

So, I am always at some index in a list iterator, or any iterator, so you can remove that. And I can go backwards because there is a sequence now. So, as we said, an iterator just goes through all the elements in a collection. If there is no natural order in that collection, I still have to be able to go through all through. But there may not be a meaningful way to go back to the previous one, because maybe the order in which is generated does not keep track of which order.

So, I do not keep track of what I have seen before in any particular order. So, going backwards normally may not make sense in an iterator. But in a sequence, there is always a sensible way to go back. So, a list iterator in addition to a next has a previous and just like an iterator, if you try to go next after the last element, it will be a problem.

So, in going previous before the first element is a problem, so just like hasNext you have a hasPrevious. So, in this way, this list interface allows you to exploit the fact that this ordered collection has positions, so you can make changes at fixed positions. And you can also get this more functional iterator which allows you to go forwards and backwards.

(Refer Slide Time: 06:15)



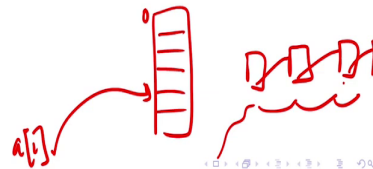
The List interface and random access

- Random access is not equally efficient for all ordered collections

- In an array, can compute location of element at index i
- In a linked list, must start at the beginning and traverse i links

```
public interface List<E>
    extends Collection<E>{
    void add(int index, E element);
    void remove(int index);
    E get(int index);
    E set(int index, E element);

    ListIterator<E> listIterator();
}
```



So, though all this is provided this random access in particular, you know, getting to the i 'th element is provided to you by this list interface, it is not true that all versions of sequences are equally amenable to random access. So, if you have an array, so an array is a picture that we saw before, which is just like this, then normally, what you can do is you can if you give me i , I can calculate the position of a of i because I know that this is 0'th position.

And normally an array will have all elements of a uniform type. So, I will just have to say, okay, this is a block of storage, here is my first element, I have to skip i elements times the size of each element. And this is where the i 'th element will be. So, just by using arithmetic, I can figure out from the start of an array where the i 'th position is.

On the other hand, if I have this usual kind of LinkedList, as we saw, where each element points the next, the only way I can do it is to actually traverse i links, so it takes me time proportional to the position to reach that position. So, even though I can talk about the i 'th element in the LinkedList, it is still a meaningful concept because the list has a beginning and an end and it has position 0, 1, 2, 3. Getting to the i 'th element is no longer the same in terms of efficiency. So, the functionality is the same, but the efficiency is not the same.

(Refer Slide Time: 07:36)



The List interface and random access

- Random access is not equally efficient for all ordered collections
 - In an array, can compute location of element at index i
 - In a linked list, must start at the beginning and traverse i links
- Tagging interface **Random Access**
 - Tells us whether a **List** supports random access or not
 - Can choose algorithmic strategy based on this

```
public interface List<E>
    extends Collection<E>{
    void add(int index, E element);
    void remove(int index);
    E get(int index);
    E set(int index, E element);

    ListIterator<E> listIterator();
}

if (c instanceof RandomAccess) {
    // use random access algorithm
} else {
    // use sequential access algorithm
}
```



Madhavan Mukund

Concrete Collections

Programming Concepts using Java

4/10

So, what we can use is so, there is this, this should be access. So, there is a tagging interface called random access. So, random access is it does not do anything, it is a special kind of interface, which is there we will see in other places also in Java, it just is a way of indicating whether a class has as a whole has a property or not. It can object has a property or not. So, we can say if c , the thing that I am dealing with, is an instance.

So, this collection is an instance of random access, then I can assume that random access methods are actually efficient. So, if I have a designed an algorithm, which wants me to go to the i 'th element in the j 'th element, so a typical example is a swap kind of thing. If I want to swap the i 'th element in the j 'th element in a collection, if I have random access is very easy, because getting to the i 'th element getting to the j 'th element takes the same amount of time, a constant amount of time and then exchange them.

So, I just reassign these two values. Whereas if I have to go to the i 'th element, go to the j 'th element and then update them, then I have to first go through once and pick up the i 'th value, pick up the j 'th value, then had to go back again, and then go back to the i 'th value and put the value that I found at position j and so on. So, I have to actually go around and around this list more than once.

So, I can choose a strategy by looking up this tagging interface called random access. So, that is one way of getting around this problem that the list interface as such, provides you random

access without any promise that the underlying implementation. Remember list is still an interface, it is only telling you capabilities.

So, as anything, which is an actual object, which implements or extends this interface list will have the capability of going to the i'th element. Now, whether it is a meaningful and efficient thing to do with that data structure or not, you can find out at runtime through this random access.

(Refer Slide Time: 09:24)

The AbstractList interface

- Recall that `AbstractCollection` is the "usable" version of `Collection`
- Correspondingly, `AbstractList` extends `AbstractCollection`
 - Inherits default implementations
- `AbstractSequentialList` extends `AbstractList`
 - A further subclass to distinguish lists without random access
- Concrete generic class `LinkedList<E>` extends `AbstractSequentialList`
 - Internally, the usual flexible linked list
 - Efficient to add and remove elements at arbitrary positions

Madhavan Mukund Concrete Collections Programming Concepts using Java 5/10

So, remember, we said that collection as it stands is not very usable, because it has this whole zoo of abstract functions which need to be implemented and it will be a real tedious task to implement all of them. So, instead, Java created this intermediate abstract class or abstract collection, which sits below collection and provides default implementations for many things. So, in a similar way, list interface that we talked about is a sub interface of the original collection, but there is a corresponding version of the list interface which is called abstract list which sits under abstract collection.

And this abstract list will have all the default implementations that come from abstract collection. So, abstract list will inherit everything that abstract collection does, but it will also have all the extra interface capabilities that list has. So, it is kind of a merge of the two. And then within this, there is this notion of random access and not random access. So, we have a further subclass, or sub abstract class called abstract sequential list, which extends this abstract list class.

And this is for list implementations that do not support random access. So, if we look at concrete classes that are available in Java itself, these are not things that we have to write. So, there is something called a `LinkedList`. So, `LinkedList` is exactly what you would expect in something that implements this, except that you do not have to worry about the implementation. So, normally, when we write in a programming kind of course, we are asked to implement a `LinkedList`, is our job to keep track of these nodes and all that. This is something which is provided to us.

So, Java has provided us a `LinkedList`, it keeps track of the nodes we do not worry about. And it has all the functionality that we get with the interface. So, we can traverse it using an iterator, we can get a list iterator, we can go backwards, forwards, all that. So, at a high level, it is just like any other list, the fact that it is implemented as a `LinkedList` is just information for us to know how to use it effectively. So, whether or not is an efficient thing to use or not.

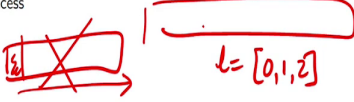
So, internally it is just the usual flexible `LinkedList`. So, in a `LinkedList`, the usual thing that is good is if I want to remove this element, then it is relatively easy, I just have to do some plumbing. And if I want to insert an element here, I just have to change again, this link and do it. So, inserting and deleting and modifying a list in the middle is relatively inexpensive with a `LinkedList`. But what is expensive with a `LinkedList` is getting to a particular position. If I want to go to the i 'th position, I have to walk there. So, there is a trade-off between flexibility and this random access.

(Refer Slide Time: 11:53)



The AbstractList interface

- Recall that `AbstractCollection` is the "usable" version of `Collection`
- Correspondingly, `AbstractList` extends `AbstractCollection`
 - Inherits default implementations
- `AbstractSequentialList` extends `AbstractList`
 - A further subclass to distinguish lists without random access
- Concrete generic class `LinkedList<E>` extends `AbstractSequentialList`
 - Internally, the usual flexible linked list
 - Efficient to add and remove elements at arbitrary positions
- Concrete generic class `ArrayList<E>` extends `AbstractList`
 - Flexible size array, supports random access



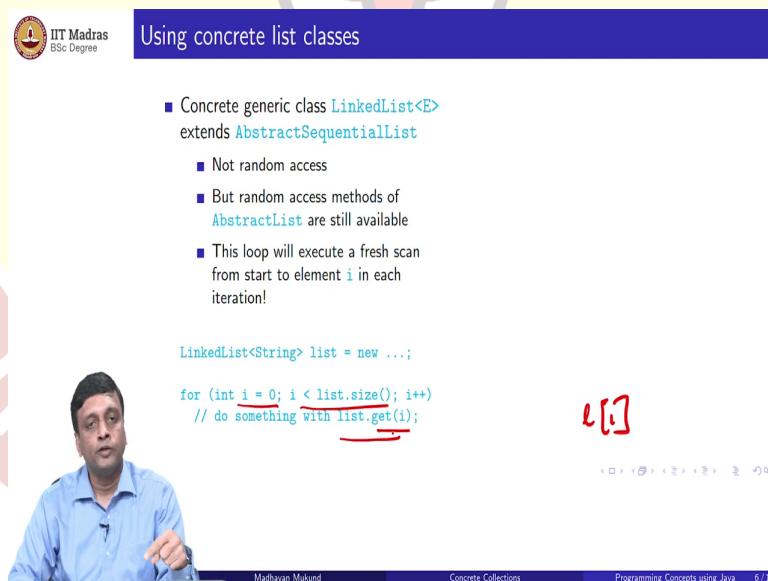


Madhavan Mukund Concrete Collections Programming Concepts using Java 5/10

Similarly, there is something called an ArrayList. So, an ArrayList is essentially an array, except that its size is not fixed. So, when we write a Java array, we have to say this is an array of size something. An ArrayList, we do not have fixed sizes, it is an ArrayList of a particular type and it will just keep growing as needed. So, it is a flexible size array, but the advantages it supports, so underlying it is an array. So, actually, the list that we see in Python are actually array list.

So, in Python, when you say l is equal to whatever, 0, 1, 2, it is not that Python is creating a list with exactly three elements, what Python does is actually gives us a large block of storage is an array and it uses a small part of it. And then if our list actually exceeds this, then it does goes ahead and says, okay, this is not good. So, it gives me a double size storage and say, henceforth work with this. So, as the list expands in size, the array allocated will change, and I will get a bigger array, but at any given point, I am actually working within an array. So, random access is possible. So, that is how Python list work. And that is how an ArrayList works.

(Refer Slide Time: 12:59)



IIT Madras
BSc Degree

Using concrete list classes

- Concrete generic class `LinkedList<E>` extends `AbstractSequentialList`
 - Not random access
 - But random access methods of `AbstractList` are still available
 - This loop will execute a fresh scan from start to element `i` in each iteration!

```
LinkedList<String> list = new ...;  
for (int i = 0; i < list.size(); i++)  
    // do something with list.get(i);
```

Madhavan Mukund Concrete Collections Programming Concepts using Java 6/10

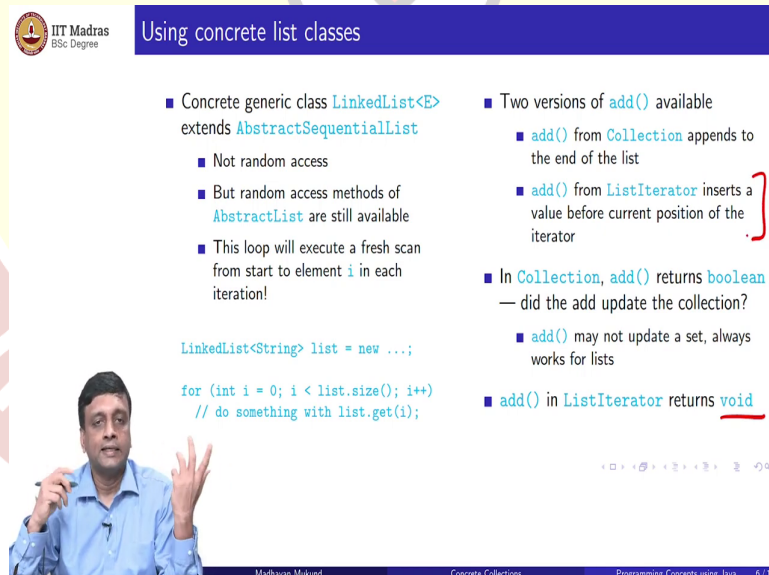
So, this concrete class, as we said, LinkedList extends abstract sequential lists, and therefore, it is not a random-access class. But remember that this is only a kind of limitation from the point of view of efficiency. The random-access methods are still available, you are not prevented from using random access methods, because that is part of the list interface. So, for

instance, you could well write a loop like this. You can say, for i equal to 0 and up to the list of the size, get the i 'th value. So, this is, remember that get is this function.

And what will this do? Actually, it is kind of strange. Normally, when you are doing something like this, if you were doing it by hand, for instance, you would actually go through this list once, so you will go to the 0'th element. So, you could actually write a loop like this to go through a list manually, efficiently by going through it once. But because we are using this kind of get operation, what will happen here is that each time I call get, it will start again from the beginning and try to go to the i 'th one, so it will try to simulate this random access.

So, it is not remembering where it was, it does not know that it was at i minus 1 in the previous step, and it just goes back and from scratch, it goes from 0 to i , so this is horrendously slow, but use, it is not forbidden. So, this is something that you keep in mind. So, the fact that these things are known to be sequential lists, does not forbid you from using the random-access functions in this. So, it is up to you as a programmer to make sure that you are not using the wrong function on a particular data structure.

(Refer Slide Time: 14:35)



IIT Madras
BSc Degree

Using concrete list classes

- Concrete generic class `LinkedList<E>` extends `AbstractSequentialList`
 - Not random access
 - But random access methods of `AbstractList` are still available
 - This loop will execute a fresh scan from start to element i in each iteration!
- Two versions of `add()` available
 - `add()` from `Collection` appends to the end of the list
 - `add()` from `ListIterator` inserts a value before current position of the iterator
- In `Collection`, `add()` returns `boolean` — did the add update the collection?
 - `add()` may not update a set, always works for lists
- `add()` in `ListIterator` returns `void`

```
LinkedList<String> list = new ...;  
for (int i = 0; i < list.size(); i++)  
    // do something with list.get(i);
```

Madhavan Mukund Concrete Collections Programming Concepts using Java 6/10

The other thing to keep in mind is that we have sometimes multiple versions of similar looking functions. So, we have this add function which is available at the level of the collection. We can provide a new element to a collection as a whole. We also said that the list iterator allows us to add a value or insert a value rather before the current position of the list.

So, this is not there in the normal iterator. The normal iterator allowed a remove. But this is allowing us an add. So, this is a different add from the add, which is available in collection.

So, the add which is available in collection is like an append. It will add at the end of the list. The add which is available in the iterator is actually an insert, it will insert at the current position. There also a difference in the signature, the collection add returns a Boolean. You might wonder why should it return a Boolean? Well, it returns a Boolean, because in certain collections add may not work. Remember that we said that a set has no duplicates.

So, if I try to add a value *v* to a set, and the value *v* is already present in the set, then add has no effect. So, really, this add reports to you whether the set changed or not, whether the collection changed or not. Now, of course, in the context of a set, this may or may not happen. In the context of a list, it will always happen.

So, if I use this collection version of add for a list, I will always get a response true, if I use it for a set, I may or may not get the response true. So, this is the version of add which is there in collection. It adds at the end, the version of add which is there in the iterator does not have to worry about sets, because it is not something which is associated with unordered collections or with collections without duplicate, it is specific to these ordered collections, where duplicates are allowed.

So, insert will always work in this case. So, therefore there is no need to signal whether it is true or false. So, therefore, the add in the list iterator actually returns a void, so they actually have different signatures. But it is just important to remember that when you are dealing with a list, do not accidentally.

Because they occur in two different `(( ))`(16:45), explicitly create an iterator and use that add to get this insert operation. Whereas if I just have a list *l*, and I say *l.add()*; I will end up using the collection one, which appends at the end. So, it is important to be aware of the subtle differences between the different versions of the function which have similar names in this thing.

(Refer Slide Time: 17:03)

The Set interface

- A set is a collection without duplicates
- `Set` interface is identical to `Collection`, but behaviour is more constrained
 - `add()` should have no effect, and return `false`, if the element already exists
 - `equals()` should return `true` if contents match after disregarding order
- Two interfaces, same signature?
- Use `Set` to constrain values to satisfy additional constraints
- `Set` implementations typically designed to allow efficient membership tests
- Ordered collections loop through a sequence to find an element
- Instead, map the value to its position
 - Hash function
- Or arrange values in a two dimensional structure
 - Balanced search tree
- As usual, concrete set implementations extend `AbstractSet`, which extends `AbstractCollection`



Madhavan Mukund

Concrete Collections

Programming Concepts using Java

7/10

So, let us move from ordered collections to unordered collection. So, we have something called a set. So, like list set in itself is an interface and what set guarantees use the collection without duplicates. So, set has unlike list set, so remember list added something to the collection interface. It added these random-access things, which allowed you to add, get things at particular value set things that particular position, so on.

So, a set does not have any other things. So, the interface of set is actually the same as the interface of collection. So, the only thing is that the behaviour of the functions is constrained. So, in a set, add should have no effect and should return false, equals should return true if the collection as a whole is the same, even though there might be differences in order. So, we have more constraints, and you might ask, what is the point of having two interfaces is the same signature, it is so that as a programmer, you can insist that an argument to a particular function must be a set and not an arbitrary collection.

So, it allows you to constrain the type, to behave within constraints, which you can then assume in the code that you are writing. So, you know that when something passes this set type declaration in your function that you can operate on it as though it is a set. So, that is the reason why this, it is important to keep these two names differently, even though they look superficially, they will have the same functions available to them.

So, the principal difference between implementing a set, I mean, you could of course, implement a set inefficiently by just calling it a collection of ordered size, and every time we want to insert, you could just walk through the entire collection and see if the value is there.

And if it is not there, add it if it is there, do not add it. So, there is no problem implementing a set using a list or an array.

But the difficulty is that this is not the intention. The intention of a set is that you want to quickly check whether something is there or not. So, a set is really used to keep track of a collection of things and add things if it is not there and remove things if it is there and so on. So, as we said, if you have an ordered collection, the only way to do this is actually cycle through the entire collection, you have to do a kind of iterator type of thing or you have to go through the whole thing.

I mean of course if it has some other properties like if it is sorted and if it is a random-access thing, maybe you can use binary search to find it, you can have the, you can look for the midpoint and the midpoint and so on. But still, it requires some work to do it. Whereas in a set you would like to do it upfront. So, usually, the way that a set would do this is to, it will have some underlying storage. So, every piece of data in your set is stored somewhere.

What you want is a way to take the value that is being stored and map it to that position. You want to say, if the value is 7, it must be here; if the value is 22, it must be there. And this is something which is done by applying a function, a mathematical function to the value to give you a number. And that number tells you where the storage is. And this is something which is usually called a hash function, which we will not talk about too much here.

The other way to do this is to arrange it in a nice data structure. So, there is something called a search tree, where you arrange things cleverly so that you can basically find things fast. So, this is the equivalent to what I was talking about earlier, if you have a sorted sequence, like if you have a, say, if you take a sequence like a normal dictionary, not the Python dictionary, but an English language dictionary, if you are searching for a word, the words are in alphabetical order.

So, if you can look at the word that you are looking for, you can easily decide whether it is going to be in the middle, the beginning, or the end. And if you open the dictionary at a page at random, and you look at the words on that page, you know whether the word you are searching for is before or after. So, in a very short amount of time, you can zero in on the page where the word is. So, the same way, you can kind of use this balanced search tree in order to look for a value quickly.

So, as usual, we have this set, which is an interface under collection, and it has all the abstract functions that collection defines. And we said it has exactly the same things. And then we will have a corresponding version called abstract set, which inherits all the default implementation from abstract collections when we actually provide a concrete data structure which implements this set interface, which will be actually an extension of the abstract set and not this set.

(Refer Slide Time: 21:32)

Concrete sets

- **HashSet** implements a **hash table**
 - Underlying storage is an array
 - Map value v to a position $h(v)$
 - If $h(v)$ is unoccupied, store v at that position
 - Otherwise, **collision** — different strategies to handle this case

Diagram illustrating a hash table structure. An array of slots is shown. One slot at index k contains value v . Another value v' is shown with an arrow pointing to the same slot k , illustrating a collision. Handwritten notes include v , $h(v)=k$, v' , and $h(v')=k$.

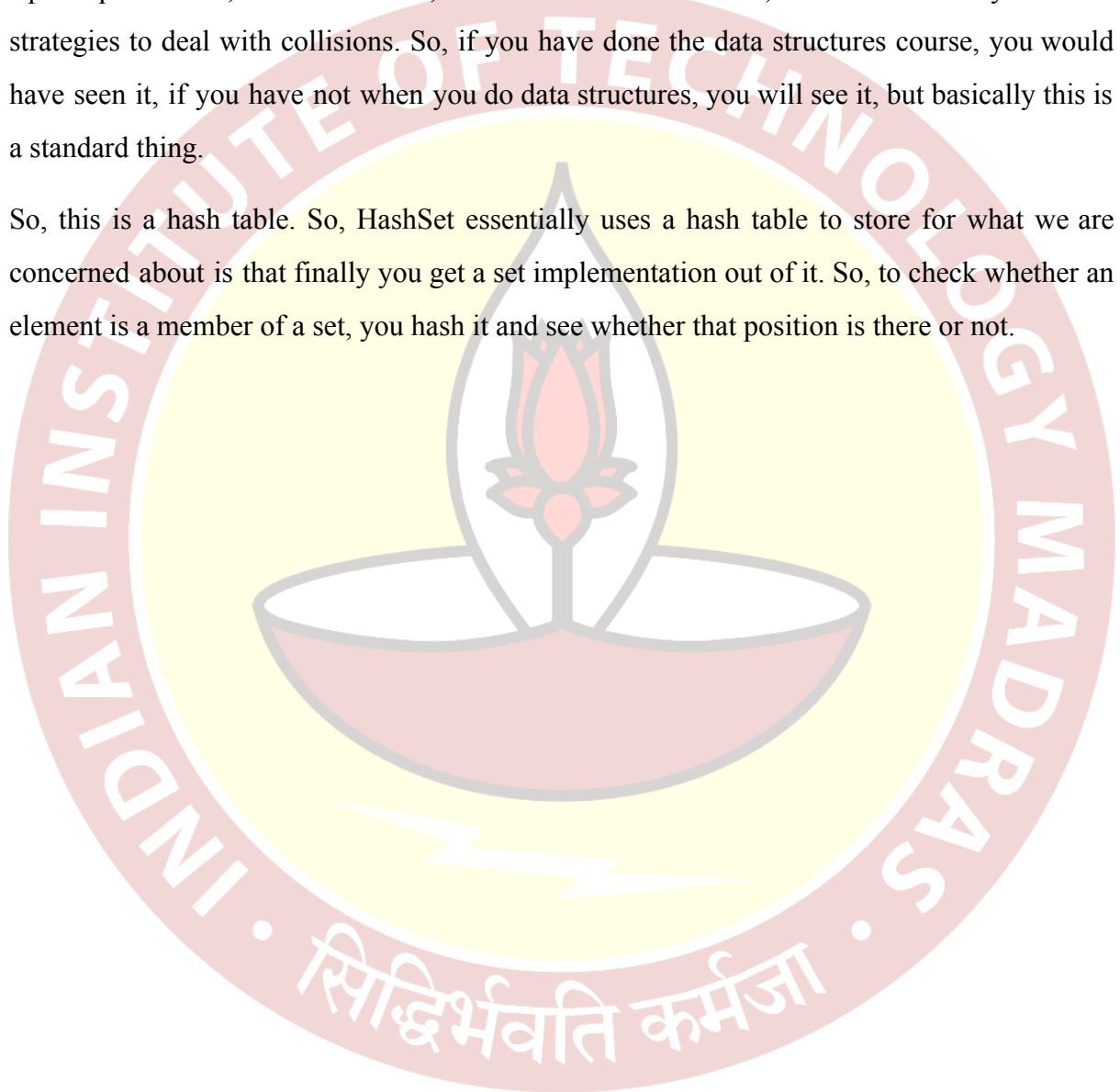
So, what are the concrete things? Well, there is something called a HashSet. So, HashSet is what we were talking about, it is something which implements this idea of a hash table. So, a hash table is basically I start with an array, which has some positions from 0 to n minus 1, and then some value comes. So, I compute some h of v and this gives me some k . So, this function can be something it could I mean, this is a simple thing, it could just be if it is a string, it could be for example, if I take the bits in the string and I just think of it as a binary number, and then I do modulo n or something like that.

It is a some function, we are not worried about. The design of this function is important for this data structure to work well, but let us not worry about, abstractly it gives you a case. So, I go to position k and I put my v here. Now, this is fine, except that now I give you another, so remember that this is a finite thing, 0 to n minus 1 and there might be an arbitrarily large range of values which come.

So, more than one value will have to be mapped the same thing because I cannot have storage, which is equal to all the strings in the universe for example, that would be impossible. So, now another v prime comes and it happens at h of v prime is also equal to k . So, I go to the same k and I find that there is already a v there, then I have to do something.

So, one possibility is I go down and I say, okay, is the next slot free, and then if I use that, and I put v prime here, so these are the, so this is called a collision, and there are many different strategies to deal with collisions. So, if you have done the data structures course, you would have seen it, if you have not when you do data structures, you will see it, but basically this is a standard thing.

So, this is a hash table. So, HashSet essentially uses a hash table to store for what we are concerned about is that finally you get a set implementation out of it. So, to check whether an element is a member of a set, you hash it and see whether that position is there or not.



(Refer Slide Time: 23:17)



Concrete sets

- **HashSet** implements a **hash table**
 - Underlying storage is an array
 - Map value v to a position $h(v)$
 - If $h(v)$ is unoccupied, store v at that position
 - Otherwise, **collision** — different strategies to handle this case
- Checking membership is fast — check if v is at position $h(v)$.
- Unordered, but supports **iterator()**
- Scan elements in **unspecified** order
- Visit each element exactly once
- **TreeSet** uses a tree representation
 - Values are ordered
 - Maintains a sorted collection
- Iterator will visit elements in sorted order
- Insertion is more complex than a hash table
 - Time $O(\log n)$ if the set has n elements



Madhavan Mukund

Concrete Collections

Programming Concepts using Java

8 / 10

So, checking membership is fast, you just check whether v is at $h(v)$ module or this collision and looking for it elsewhere. So, this is unordered. There is no particular sequence as we said this hash function kind of scrambles the values according to that thing. So, there is no such thing as the first element in the set and the last element in the set and so on. But of course, the notion of an iterator still makes sense.

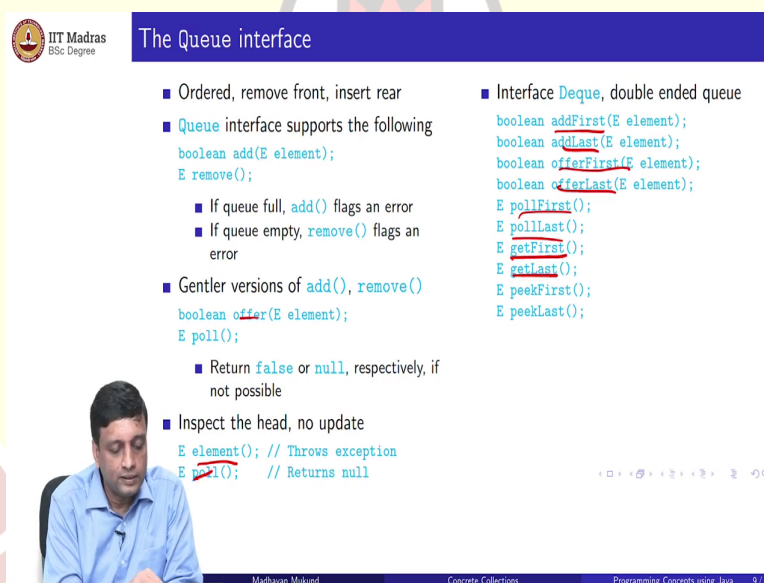
So, if I ask for, because it is a collection after all, so it must support an iterator. So, if I ask for an iterator and I start somewhere, what will essentially happen is that I will get all the elements in the set in some unspecified order, I cannot predict the order in which they will come. But I am guaranteed that if I start with a fresh iterator, and I keep saying until as long as `hasNext` true, I keep getting next and I keep going until `hasNext` becomes false if I run through that usual while loop involving, or if I say for something element x colon s , if I do any of the standard things that we do with an iterator, I am guaranteed that I will visit every element in that set exactly once.

So, in that sense, I will loop through the set but not in any fixed order. So, the iterator still makes sense to examine all the elements in a set, even though it does not have any fixed order. So, we said that `HashSet` is one thing we also talked about this search trees, so there is a another `TreeSet` which is a tree representation. So, one assumption that we need when we do a tree representation is the values in our set can actually be compared, I can say this value is bigger than that value that bigger.

So, really what happens is that this TreeSet maintains a sorted collection. And since it is sorted, actually we have extra structure because we have a smallest element with respect to the sorted order and largest element. So, actually in such a situation, unlike in the HashSet, the iterator actually will go through the values in sorted order. So, we have an extra guarantee.

So, again, you may not need to worry at the level of set whether using HashSet or a TreeSet, but TreeSet has more functionality. So, if you need this, then you can use it, but it comes with a price, which is that maintaining a tree is more complicated than maintaining a hash table. So, each entry insert and delete will take time proportional to the log of the number. So, you pay a price on one side and you gain a benefit on the other side. You have an ordered iterator but you pay more for each insert into this tree.

(Refer Slide Time: 25:42)



The Queue interface

- Ordered, remove front, insert rear
- Queue interface supports the following
 - `boolean add(E element);`
 - `E remove();`
 - If queue full, `add()` flags an error
 - If queue empty, `remove()` flags an error
 - Gentler versions of `add()`, `remove()`
 - `boolean offer(E element);`
 - `E poll();`
 - Return `false` or `null`, respectively, if not possible
 - Inspect the head, no update
 - `E element();` // Throws exception
 - `E peek();` // Returns null
- Interface Deque, double ended queue
 - `boolean addFirst(E element);`
 - `boolean addLast(E element);`
 - `boolean offerFirst(E element);`
 - `boolean offerLast(E element);`
 - `E pollFirst();`
 - `E pollLast();`
 - `E getFirst();`
 - `E getLast();`
 - `E peekFirst();`
 - `E peekLast();`

Madhavan Mukund Concrete Collections Programming Concepts using Java 9/10

So, finally, let us look at this queue remember a queue is an ordered sequence in which you insert at the back and you remove from the front. So, this would be a queue interface. So, it has this add and remove. And now, a queue will typically have a bound, so add may not always be possible. So, if the queue is full, if the queue may not have a bound, it might be a LinkedList in which case the queue will always allow you to add, but if there is a size limitation in the queue, like you have implemented a queue in an array, for example, and you run out of space, then add will actually flag an error, we have not talked about errors in Java, we will do that probably soon in the next week or so.

But you will formally get an array say error saying that you try to add to queue which had no space. More importantly, we will try to remove an element from a queue which is empty, that is also an error because there is nothing to remove. So, these are the things that happen when you use add and remove.

So, what queue does is it gives you a kind of gentler way of doing this, has the same functionality. So, instead of saying add you can say offer and instead of saying remove you can say poll, so if it work it works in same way. So, offer is the same as add, remove is the same as poll provided the operation succeeds, if the operation fails on the other hand, instead of getting a Java error, which you have to handle through this some error handling thing, you will get a value.

So, here if I try to add and it fails to add, then the add will return a Boolean if it is true, it means the add succeeded; if it is false, it means the add failed. Similarly, if I poll and there is nothing in the queue, instead of getting an error, I will get a null. So, if I get back nothing from my poll, then I know that there was nothing there. And there is also a way of inspecting the head of the queue without removing it. So, we just want to see what is the first element in the queue.

So, this should, should be I think peek. So, there is something called element which we will look at the first element but again, we will throw an error, and there is something called peak, which we will look at the first element and if it is not there, if there is nothing there, it will return a null. So, this is the basic queue interface. So, this is again an interface, not an implementation.

Now, we also talked about when we were discussing this difference between subtyping and inheritance, we talked about this thing called Double Ended Queue. So, Double Ended Queue allows these things to happen at both ends, you can have a insert at the beginning insert at the end. So, you take all these functions.

So, you take add and you make an addFirst addLast. You take an offer and you make an offerFirst, offerLast. So, beginning and end. Poll, you can do a pollFirst, pollLast. And then here instead of element, it is called get, you have a getFirst, getLast. And remember that this was peak, you have a peakFirst, and peakLast. So, is essentially a double ended version of the interface of queue. So, you can do this thing at both ends.

(Refer Slide Time: 28:41)



The Queue interface

- Ordered, remove front, insert rear
- **Queue** interface supports the following
 - `boolean add(E element);`
 - `E remove();`
 - If queue full, `add()` flags an error
 - If queue empty, `remove()` flags an error
 - Gentler versions of `add()`, `remove()`
 - `boolean offer(E element);`
 - `E poll();`
 - Return `false` or `null`, respectively, if not possible
 - Inspect the head, no update
 - `E element();` // Throws exception
 - `E poll();` // Returns null
- Interface **Deque**, double ended queue
 - `boolean addFirst(E element);`
 - `boolean addLast(E element);`
 - `boolean offerFirst(E element);`
 - `boolean offerLast(E element);`
 - `E pollFirst();`
 - `E pollLast();`
 - `E getFirst();`
 - `E getLast();`
 - `E peekFirst();`
 - `E peekLast();`
- Interface **PriorityQueue**
 - `remove()` returns highest priority item
- Concrete implementations
 - **LinkedList** — implements **Queue**
 - **ArrayDeque** — circular array **Deque**



Madhavan Mukund

Concrete Collections

Programming Concepts using Java

9 / 10

And then there is another type of a queue which is called a priority queue. So, essentially in a normal queue, the priority is in order of arrival. So, the thing that arrived first has the highest priority. So, people are served in a normal queue like you go to a shop or some kind of place. People are served in the order in which you arrived, but what if you had a kind of service, where people who pay more get served first.

So, people have different types of tickets. So, they buy and now if they have a ticket for which you have paid more you jump the queue, so you have a priority. So, this is called a priority queue. So, in a priority queue remove will, remove not the earliest value that came which is what you will get in a normal queue, but the highest priority. So, each element in the queue, they will have a value associated, it will also have a priority and you will choose according to that. So, these are all different types of queues and these are all interfaces.

And concretely, you can use some very basic data structures like a **LinkedList** is in implicitly a queue. And when you insert you inserted the tail and when you remove you remove from the head. So, a **LinkedList** on its own with just the add in the normal, add operation at the end and the remove the first thing becomes a queue. You can take the circular array that we talked about earlier. For queue you can have same interpreted for a deck.

So, you remember the beginning and the end, and then you can insert and delete at both ends rather than just inserting at one end and deleting it on another end. So, there is a built-in type in Java called **array index**. So, this is not exhaustive by any means. So, you should look at the

Java documentation and if you are trying to look for a particular data structure, if you know about a data structure, which is useful for you, very likely there is an implementation already in Java.

So, make sure that you check before you try to implement a new data structure. But that is one of the nice things about Java, which has a large collection of standard data structures, which are defined in data structures, textbooks and so on, are actually already available as implementations and they all fit into this collection hierarchy. So, that is something that you should keep in mind.

So, you should try to understand how these interfaces are all organized. So, that there is a natural, I mean, it is a bit like the way that biologists try to group species. So, it is a bit like all these things have been classified, there is what is called a taxonomy of these collections, and they are all fitted into a tree so that they all sit in their appropriate place according to their capabilities.

(Refer Slide Time: 31:08)

Summary

- Different types of **Collection** are specified by subinterfaces
 - **List, Set, Queue**
- **List** allows random access, more functional **ListIterator**
- **Set** constrains collection to not have duplicates
- **Queue** supports restricted add and remove methods
- Each interface has corresponding version under **AbstractClass** *Collection*
- Concrete implementations extend **AbstractList**, **AbstractSet** and **AbstractQueue**

Madhavan Mukund Concrete Collections Programming Concepts using Java 10 / 10

So, to summarize, we have these different types of collections which are defined by sub interfaces. So, ordered collections are of type list, unordered but without duplicates are of type set, not necessarily unordered, but certainly without duplicates. And those which have this restricted insert, delete or under queue, so list will allow random access and it has a more functional list at iterator.

Set will ensure that there are no duplicates. Queue will support these restricted add and remove. And all of these are actually implemented by having a corresponding interface or abstract class rather, which is under abstract collection. So, they will have a corresponding version under abstract collection which implements many of the default methods for the large number of methods that they are in collection. So, the concrete implementations will extend this abstract list, abstract set, and abstract queue.

